ChatGPT

Modules Handling Workflow Execution in ComfyUI

When you click **Run** in ComfyUI, the backend goes through the **workflow graph** and gathers all the necessary parameters from each node, then executes the nodes in the correct order. The logic for this is implemented in ComfyUI's core code (which is open source). The key modules and functions involved are outlined below:

`execution.py` – The Execution Engine

The `execution.py` module in the ComfyUI repository contains the primary logic for parsing the workflow and executing it. This is where ComfyUI figures out which nodes to run and in what order, and how to collect their parameter values. In summary:

- **Identifying Output Nodes:** ComfyUI first identifies *output* nodes (like the **Save Image** or Preview nodes) which produce final results. These nodes are marked as `OUTPUT_NODE = True` in their class definitions, so the system knows they must always execute [1] [2]. When Run is clicked, the engine takes those output nodes as the starting point for execution.

- **Traversing Backward Through the Graph:** From each output node, ComfyUI works *backwards* through its inputs to find all upstream nodes that feed into it [1]. In the code, this is done either via a recursive approach or an execution scheduling structure. For example, older versions use a recursive function `recursive_execute` that calls itself on a node's prerequisites before running the node. You can see this in `execution.py`, where for each input that is a link to another node, it calls `recursive_execute` on that upstream node before proceeding [3]. This ensures that **only the nodes in the chain leading to the output node are executed** (any unrelated nodes in the workspace are ignored).

- **Scheduling Node Execution:** Newer versions of ComfyUI use a scheduler (`ExecutionList` and `PromptExecutor` in `execution.py`) to manage execution order. The logic is similar – they add all output node IDs to an execution list, then iteratively "stage" and run any node whose inputs are ready [4] [5]. Either way, the *execution engine* ensures each node runs only after its dependencies have been executed (or retrieved from cache, if already run before).

Gathering Node Parameters with `get_input_data`

As ComfyUI prepares to execute a node, it needs to **collect all input values** for that node – whether those are coming from upstream nodes or from user-specified constants. This happens in the `get_input_data` function inside `execution.py`. This function essentially *pulls out the parameters* for a node by inspecting its inputs dictionary:

- **Distinguishing Links vs. Literal Inputs:** In the workflow JSON, if an input is connected to another node, it's represented as a reference (e.g. a list like `[other_node_id, output_index]`). If it's a

direct value (like a number, text, etc.), it appears as the literal value. The `get_input_data` function checks each input entry – if it detects a reference to another node's output, it will fetch the actual output object from a cache of already-executed nodes [6]. If that upstream node hasn't been executed yet, the scheduler/recursion will trigger it first (as mentioned above). On the other hand, if the input is a literal value (and marked as required/optional in the node definition), it simply stores that value for the node's execution [6]. In short, this function assembles a dictionary of actual input values ready to pass into the node's compute function.

- **Using Node Class Definitions:** Each node type in ComfyUI is defined by a Python class with an `INPUT_TYPES` spec. The execution engine uses these definitions to know what inputs to expect. For example, it knows which inputs are *required*, *optional*, or *hidden* for the node. Hidden inputs (like a prompt ID, or extra metadata) are filled in by the system if needed [7] (e.g. ComfyUI will insert the original prompt text or image metadata if a node expects it as a hidden input). The engine looks up the node's class via a mapping (`nodes.NODE_CLASS_MAPPINGS`) in the `nodes` module to instantiate the node and read its `INPUT_TYPES` [8].

- **Executing the Node's Function:** Once `get_input_data` has gathered all the inputs, ComfyUI calls the node's execution function. Each node class defines a method (named in the `FUNCTION` property, e.g. `"invert"` or `"sample"` etc.) that actually does the work. The engine passes the collected inputs as arguments to that method and gets the outputs (usually a tuple of results matching the node's `RETURN_TYPES`) [9]. These outputs are then stored in the cache and forwarded along to any downstream nodes that need them.

Summary

In short, **ComfyUI's own code already does what we need** – it determines the subset of the graph behind a given output (such as a Save Image node) and extracts all the parameter values from those nodes to run the generation. The heavy lifting is done in the `execution.py` module: it identifies the relevant nodes (starting from the Save Image output and moving backwards) [1] [3], then uses functions like `get_input_data` to pull together every input parameter for each node before executing it [6] [9]. Essentially, when you click "Run", ComfyUI builds an execution plan that includes only the connected nodes needed for the final image and gathers their current parameter values – very much like constructing a big function call (or "command") to the Stable Diffusion model behind the scenes. Leveraging these parts of ComfyUI's code (rather than reinventing them) could indeed give us a robust way to retrieve all the settings from a workflow and replicate the image generation process.

Sources:

- ComfyUI Documentation – *Execution Control and Caching* [1] (describes how output nodes are identified and executed with their dependencies).
- ComfyUI Source – *execution.py* (`PromptExecutor` and `recursive_execute`) [3] [6] (actual code that traverses the node graph backwards and collects input values for execution).
- ComfyUI Source – *execution.py* (`get_input_data` and node parameter handling) [6] [7].
- ComfyUI Source – *execution.py* (using `NODE_CLASS_MAPPINGS` to find node classes) [8].

1 2 Properties - ComfyUI
https://docs.comfy.org/custom-nodes/backend/server_overview

3 6 7 9 execution.py · Lucas/ComfyUI - Gitee
https://gitee.com/failurejack/ComfyUI/blob/master/execution.py

4 5 8 execution.py · DegMaTsu/ComfyUI-Reactor-Fast-Face-Swap at 5471086e29d7d4973fbd41ae711ce049c64ff451
https://huggingface.co/spaces/DegMaTsu/ComfyUI-Reactor-Fast-Face-Swap/blob/5471086e29d7d4973fbd41ae711ce049c64ff451/execution.py